Blackjack in the Wolfram Language

A complete casino game, a basic-strategy coach, and a Monte-Carlo expected-value estimator, all in one package.

Marco Thiel

Abstract

This notebook presents a pure-Wolfram-Language implementation of Blackjack. It ships as two packages: `Blackjack`` contains the game engine (deck, dealer policy, scoring, payout, Hi-Lo counting, a Hit/Stand basic-strategy oracle, and a Monte-Carlo EV estimator), and `BlackjackGame`` wraps everything in a `DynamicModule` front-end with a full casino ruleset — bankroll, chip-based betting, 3:2 natural-blackjack payout, double-down, and splitting pairs (including the split-aces special case). Beyond the playable game, the notebook develops the probability theory of Blackjack: the chance of a natural ($P(\mathrm{BJ}) \approx 4.83\%$), the dealer's final-total distribution for each upcard, the Bellman formulation of the player's decision problem, the resulting basic-strategy table, and the Hi-Lo card counting system with its linear EV model and the Kelly betting criterion. Every figure in the document is produced by the same code that powers the game; the game itself can be launched from the cell at the end.

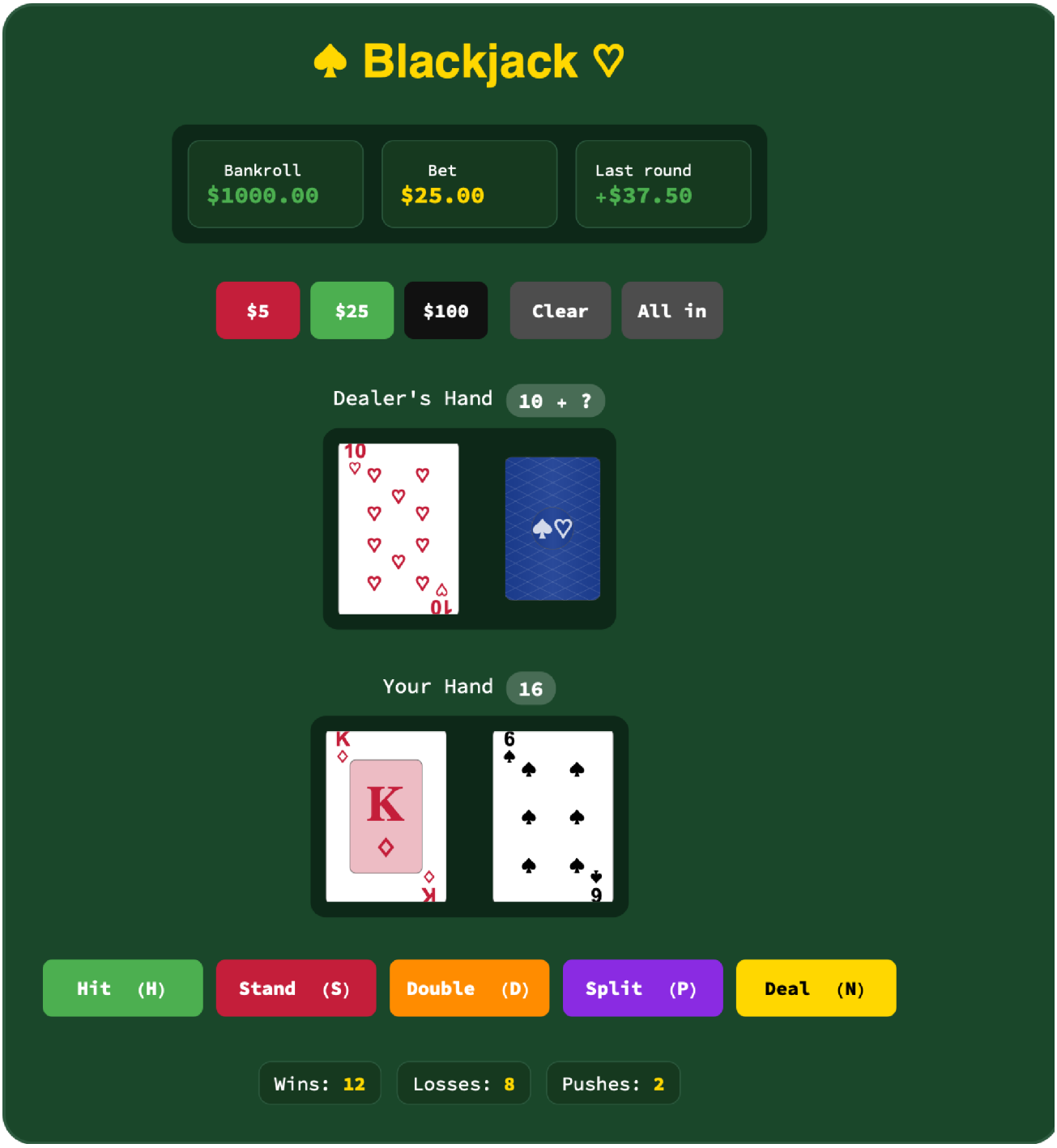


Figure 1. The Blackjack front-end (screenshot of a representative round). Bankroll / bet / last-round badges sit at the top; chip buttons adjust the wager; Hit / Stand / Double / Split / Deal sit below the hands (each has a single-letter keyboard shortcut). The Options / Coach / Shoe & card count / Session openers (collapsed here) expose S17/H17, deck-count, basic-strategy advice, Monte-Carlo EV, running count, true count, and a cumulative bankroll plot.

1. What is Blackjack?

Blackjack is a heads-up card game between a player and the house (the *dealer*). Both parties are dealt two cards from a shared shoe of one or more 52-card decks. The player's cards are face up; one of the dealer's cards is face-up (the *upcard*) and the other is face-down (the *hole card*). The goal

is to assemble a hand that **beats the dealer's total without exceeding 21**. Cards count at face value; J, Q, K count 10; an Ace counts 1 or 11, whichever gives a better total.

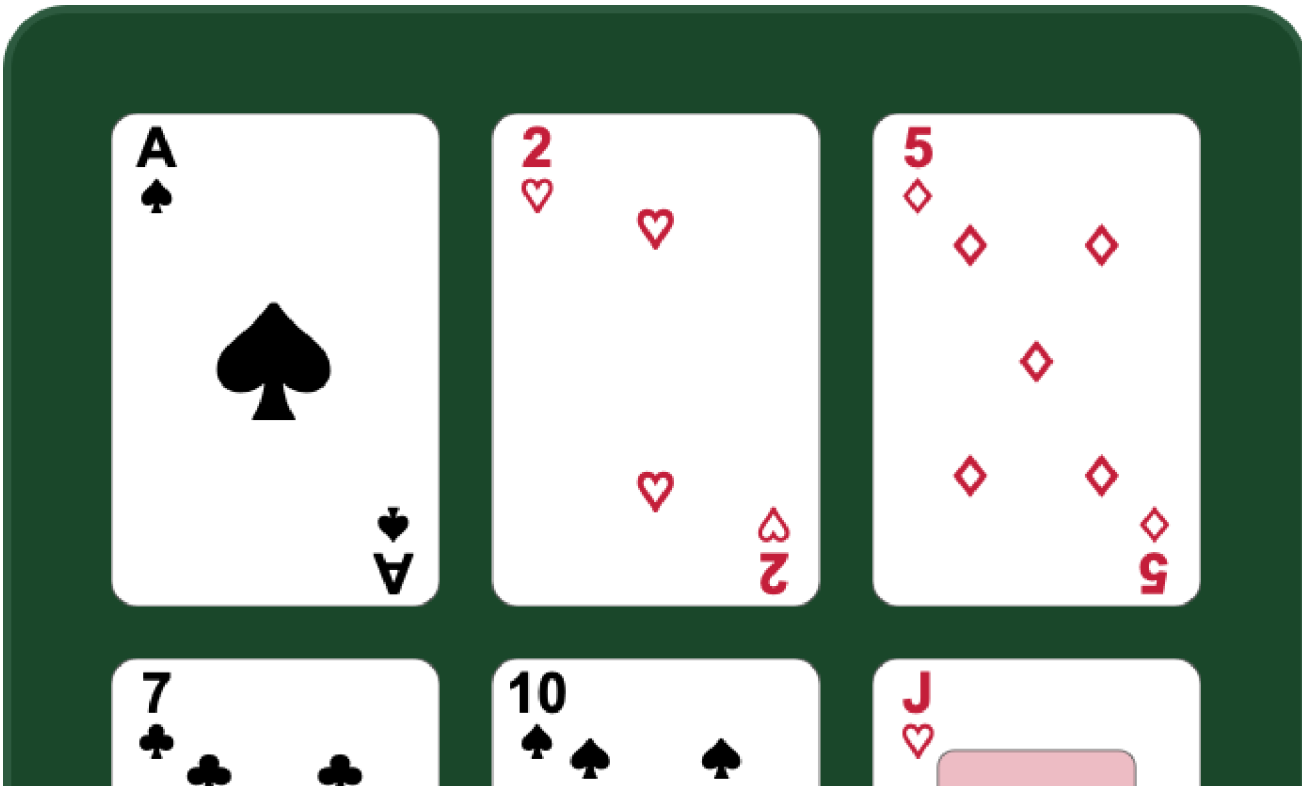
The player acts first, repeatedly choosing one of five actions:

Hit take another card;
Stand end the player's turn with the current total;
Double down double the bet, take exactly one more card, then stand automatically;
Split when the two initial cards have equal point value, split them into two hands with equal bets; split aces receive one more card each and cannot be hit further;
Surrender (not in this implementation) forfeit half the bet without playing.

Once the player stands or busts on every hand, the dealer turns up the hole card and plays a rigid policy: **hit until the total is at least 17**. Casinos differ on whether the dealer also hits a soft 17 ("H17") or stands on it ("S17"); both rules are supported here via a checkbox in the *Options* opener. A *natural blackjack* — an ace plus any ten-valued card on the initial deal — pays 3 : 2; other wins pay 1 : 1; equal totals push (0); the player loses the bet on any bust regardless of the dealer's outcome.

2. Cards and visual representation

Each card is stored as a plain association with two keys: `"value"` (one of `"A", "2", …, "10", "J", "Q", "K"`) and `"suit"` (one of the four Unicode suit glyphs). A fresh shoe is built by `CreateShoe[n]`, which cross-products ranks and suits n times and applies `RandomSample` for a uniform shuffle. Each face is rendered with pure `Graphics` primitives: a white rounded rectangle, rank+suit in the two opposite corners, and a per-rank pip layout for 2–10, a stylised letter for J / Q / K, and one large suit pip for the Ace. The back shows a blue crosshatch with a small central medallion.



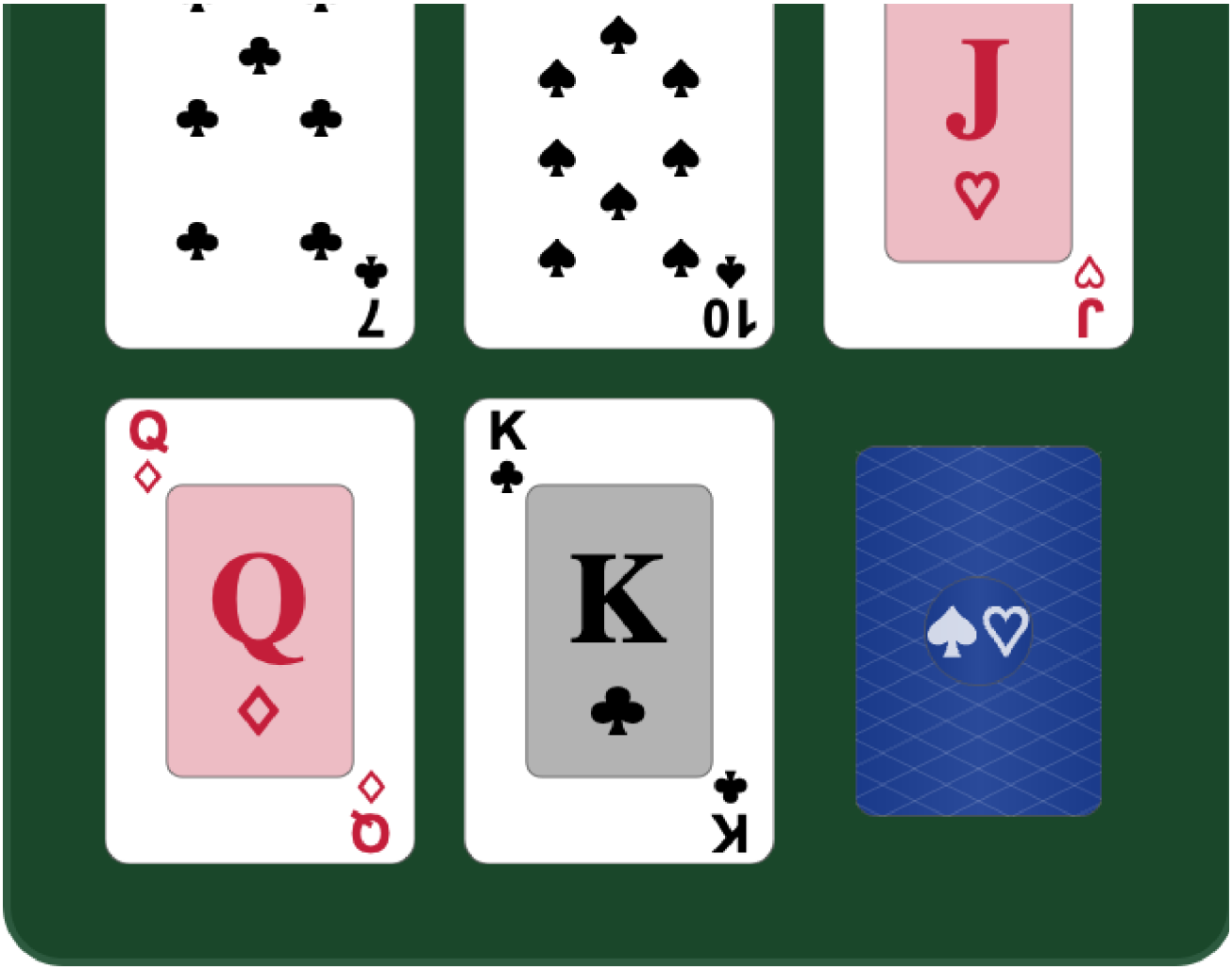


Figure 2. The eight rank templates plus the face-down card back. Hearts and diamonds render in casino red (#c41e3a); spades and clubs in black. Corners carry rank + suit so the card remains readable when fanned.

3. The mathematics of Blackjack

3.1 Probability of a natural blackjack

From a single freshly-shuffled deck of 52 cards, the probability that the player's initial two cards form a natural blackjack is

$$P(\mathrm{BJ}) = 2 \cdot \frac{4}{52} \cdot \frac{16}{51} \approx 0.0483$$

(four aces, sixteen ten-valued cards, and the order doesn't matter so we multiply by two). Blackjack's 3 : 2 payout on this outcome contributes roughly **+2.3%** to the player's expected return per hand, which is the single largest positive term in the house-edge calculation; take it away (some casinos now pay 6 : 5) and the edge roughly doubles against the player.

3.2 Dealer bust probability

Because the dealer's policy is fixed, the distribution of the dealer's final total depends only on the upcard. In the infinite-deck approximation (cards drawn with replacement from a full rank distribution) the dealer's hand is a Markov chain on (hard total, soft?)-pairs, which we can enumerate exactly by recursion:

$$P(T_{n+1} \mid T_n) = p_{\text{card}}$$

The probability of finishing bust from each starting upcard is plotted below. It is roughly U-shaped — the dealer is most dangerous showing 7, 8, 9, 10, or A (where bust probability is ~22–25%), and most vulnerable showing 4, 5 or 6 (bust probability ~40%). This single curve drives almost every asymmetry in basic strategy: the player should stand on otherwise-weak totals when the dealer is likely to bust, and press the action when the dealer is strong.

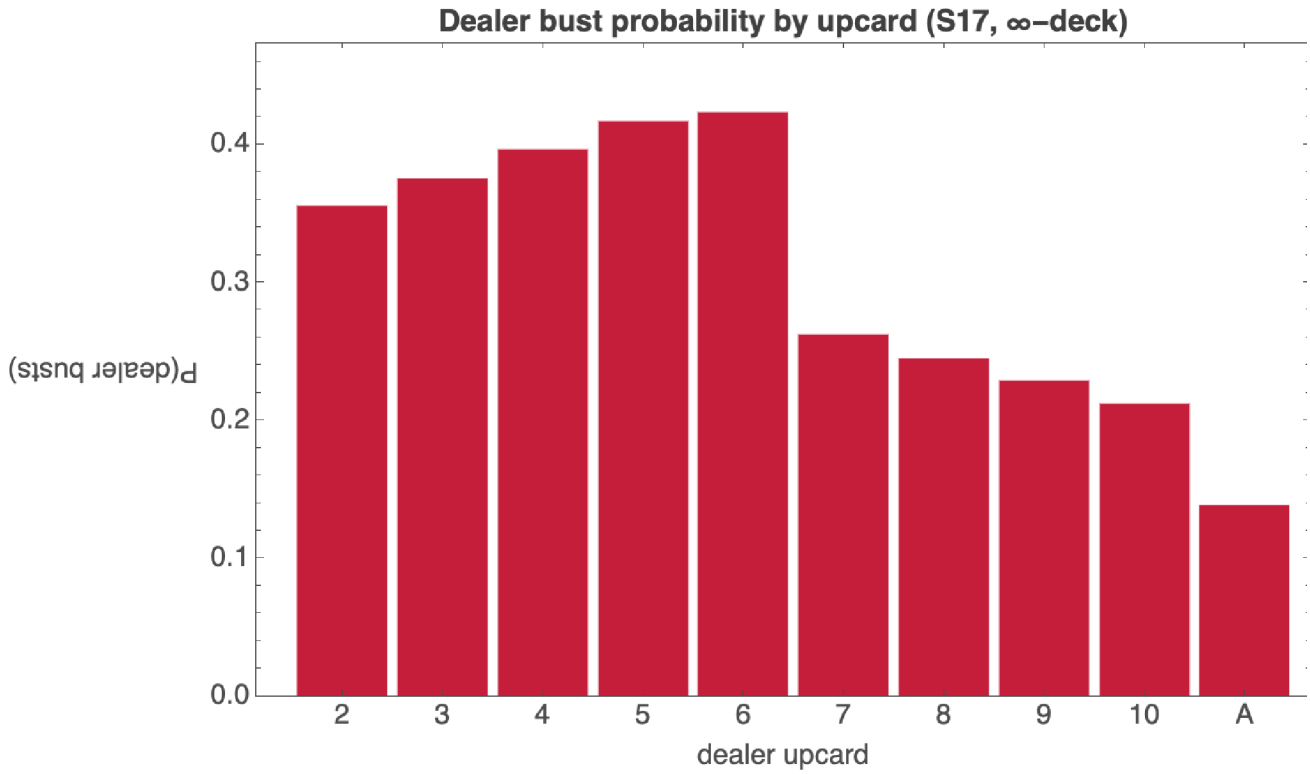


Figure 3. Exact dealer bust probability in the infinite–deck S17 model, as a function of the upcard. The valley at 4–5–6 is the reason the player stands on a hard 12–16 against those upcards.

3.3 Player decisions: the Bellman equation

From the player's seat, each state is (hand, bet, dealer upcard, remaining deck). The optimal value function satisfies the Bellman equation

$$V^*(\text{state}) = \max_a \mathbb{E}[V^*(\text{next}) \mid \text{action}]$$

taken over available actions $a \in$ {Hit, Stand, Double, Split, Surrender}. The Stand branch is closed-form once the dealer distribution is known,

$$\text{EV}_{\text{stand}}(T, u) = \sum_{T'} P(T_d = T' \mid u) \cdot \text{outcome}(T, T')$$

where $\mathrm{outcome}(T,\ T')$ is +1 (win), -1 (loss), or 0 (push). The Hit branch recursively invokes V^* on the post-hit hand; in the finite-deck case the remaining deck is smaller by one card, which is what separates the correctness of the infinite-deck basic-strategy table from a composition-dependent optimum.

3.4 The house edge

The house edge is the negative of the optimal player's expected return, computed per unit bet over a uniform distribution of starting hands:

$$\text{house edge} = -\mathbb{E}[\text{return}] \approx 0.5 \times \%$$

with standard Las Vegas rules (6 decks, S17, 3 : 2 blackjack, double-on-anything, split to four hands, late surrender allowed). Our implementation omits surrender and caps splits at four hands; these restrictions push the edge up to roughly **0.6-0.8%** for a perfect player. Notable rule variations:

H17 instead of S17: +0.22% against the player.
6 : 5 blackjack payout: +1.39%.
Single-deck vs six-deck: ~0.4% in the player's favour.
No double after split: +0.14%.

4. Basic strategy

Basic strategy is the action that maximises the Bellman value under the infinite-deck assumption, tabulated by (player total, dealer upcard). It is the correct play in the absence of any information about the shoe composition, and is within ~0.03% of perfect composition-dependent play. Because this implementation does not expose double/split in the Monte-Carlo EV routine, the basic-strategy oracle returns `H` or `S` only; double and split are still available to the human player via the buttons (and the strategy chart shows which 2-card starting hands benefit from them).

Basic strategy (Hit / Stand only)

Hard totals

ayer \	2	3	4	5	6	7	8	9	10	A
Hard 8	H	H	H	H	H	H	H	H	H	H
Hard 9	H	H	H	H	H	H	H	H	H	H
Hard 1	H	H	H	H	H	H	H	H	H	H
Hard 11	H	H	H	H	H	H	H	H	H	H

Hard 12	H	H	S	S	S	H	H	H	H	H
Hard 13	S	S	S	S	S	H	H	H	H	H
Hard 14	S	S	S	S	S	H	H	H	H	H
Hard 15	S	S	S	S	S	H	H	H	H	H
Hard 16	S	S	S	S	S	H	H	H	H	H
Hard 17	S	S	S	S	S	S	S	S	S	S
Hard 18	S	S	S	S	S	S	S	S	S	S
Hard 19	S	S	S	S	S	S	S	S	S	S
Hard 20	S	S	S	S	S	S	S	S	S	S

Soft totals

ayer \ u	2	3	4	5	6	7	8	9	10	A
ft 13 (A	H	H	H	H	H	H	H	H	H	H
ft 14 (A	H	H	H	H	H	H	H	H	H	H
ft 15 (A	H	H	H	H	H	H	H	H	H	H
ft 16 (A	H	H	H	H	H	H	H	H	H	H
ft 17 (A	H	H	H	H	H	H	H	H	H	H
ft 18 (A	S	S	S	S	S	S	S	H	H	H
ft 19 (A	S	S	S	S	S	S	S	S	S	S

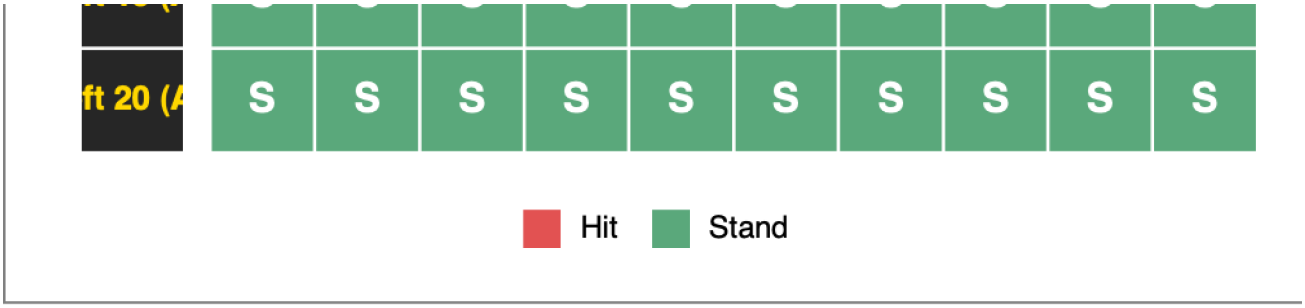


Figure 4. The Hit/Stand basic-strategy chart produced by the package's BasicStrategy function. Rows are player totals (hard on top, soft on the bottom), columns are dealer upcards. Red = Hit; green = Stand. Doubling and splitting entries of the full chart collapse to H in this reduced view — e.g. the player's 11-vs-dealer-6 cell would read "D" in a full chart but plays as H here when doubling is not permitted.

Two salient patterns in the chart: (a) the player stands on hard 12-16 exactly when the dealer upcard is 4, 5, or 6 (the valley in Figure 3); (b) soft 18 is a stand against dealer 2-8 but a hit against 9, 10, or A — the strong upcards force the player to improve even though 18 is "normally" enough.

5. Monte-Carlo expected value

For any game state the package can estimate *EV(Hit)* and *EV(Stand)* by Monte-Carlo simulation. The unseen-card pool is the remaining deck plus the dealer's hole card (since the player doesn't know it); each trial draws from this pool, plays the dealer under the current S17/H17 rule, and — for Hit — continues the player under basic strategy after the mandatory first extra card. The estimator is exposed as

```
EstimateEV[playerHand, dealerUpcard, "Hit" | "Stand", unseenDeck,
     "HitSoft17" → False, "Trials" → 600]
```

Rendering *EV(Hit) - EV(Stand)* over the player-total / dealer-upcard grid reproduces the shape of the basic-strategy chart as a continuous surface:

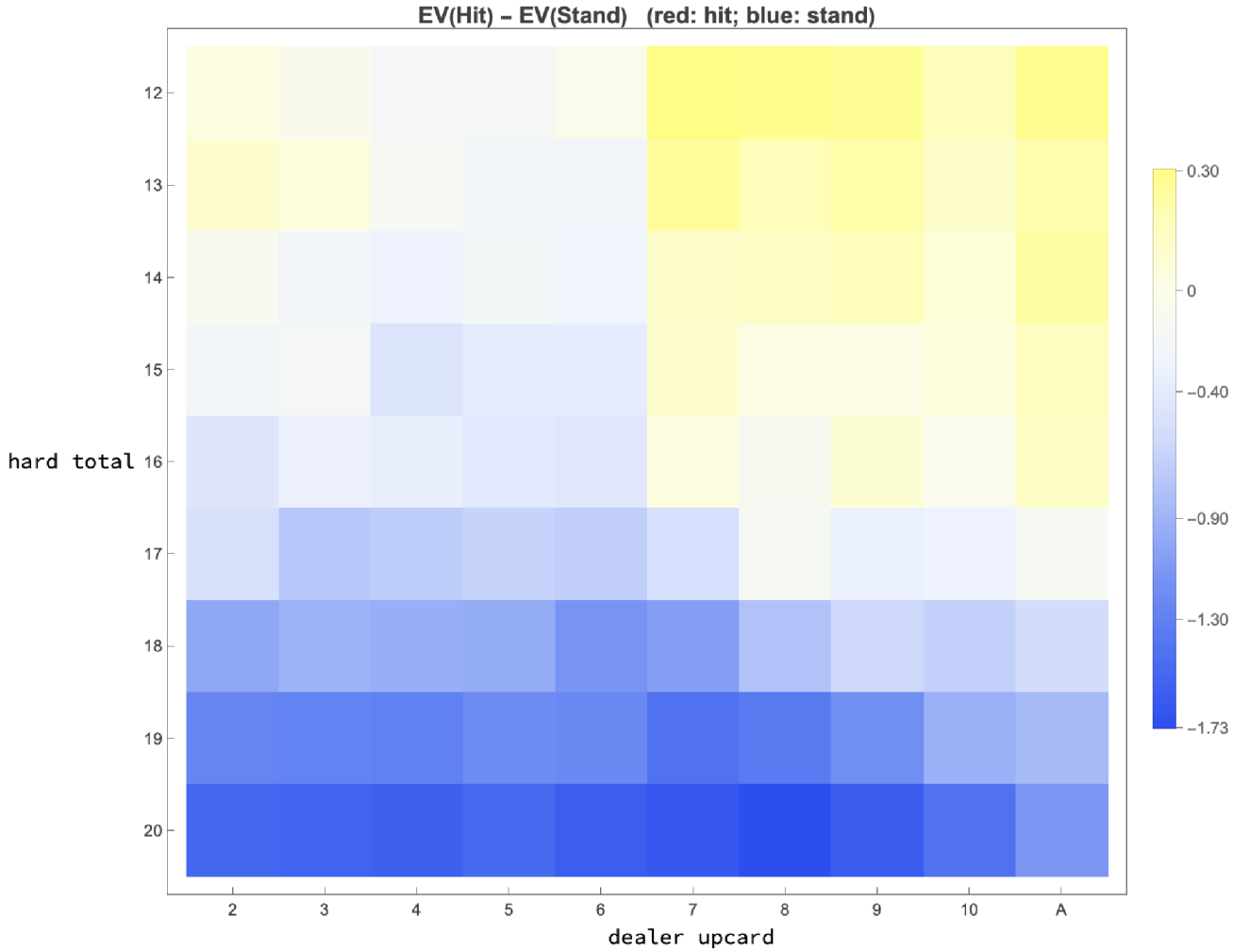


Figure 5. EV(Hit) − EV(Stand) for hard player totals 12–20 vs dealer upcards 2–A, 500 Monte–Carlo trials per cell. Red (positive) = hitting is strictly better; blue (negative) = standing is strictly better. The sharp transition at total 17 and the extra band of standing at 12–16 versus dealer 2–6 coincide with the discrete H/S boundaries in Figure 4.

6. Card counting

Basic strategy assumes every card in the shoe is equally likely to be the next drawn. As a real shoe is played out, that assumption degrades: the composition at the bottom of a six-deck shoe can look very different from a fresh shuffle. A *card-counting system* tracks a scalar running count that correlates with the player's edge. The simplest and most widespread system is **Hi-Lo**, which assigns +1 to low cards (2-6), 0 to neutral cards (7-9), and -1 to high cards (10, J, Q, K, A). When many high cards remain, the dealer is more likely to bust (Figure 3) and the player is more likely to draw a natural — both in the player's favour.

The running count RC is converted to a *true count* TC by dividing by the number of decks remaining in the shoe,

$$\mathrm{TC} = \frac{\mathrm{RC}}{\text{decks remaining}}$$

which normalises the edge signal across different shoe penetrations. The player's expected return is close to affine in TC,

$$\mathrm{EV(TC)} \approx -0.5 \times \% + 0.5 \times \% \cdot \mathrm{TC}$$

for typical six-deck rules (the slope ~0.5% per true count is an empirical regression coefficient). A classical counting bettor sizes the wager proportionally to the edge via the Kelly criterion,

$$f_* = \frac{\text{edge}}{\text{variance}}$$

capped at a practical table maximum. The game displays the live running and true counts in the *Shoe & card count* opener, and the cumulative bankroll over all rounds in the *Session* plot — so a curious player can watch their edge swing with the count over a sufficiently long session.

7. Package architecture

The code is split into two paclet-style files:

Blackjack.wl — pure game logic, no UI. Exposes `Suits`, `Ranks`, `MakeCard`, `CreateShoe`, `DrawCard`, `HandScore`, `IsSoft`, `IsBlackjack`, `IsBust`, `DealerShouldHit`, `PlayDealer`, `ResolveRound`, `Payout`, `CanDouble`, `CanSplit`, `HiLoValue`, `TrueCount`, `BasicStrategy`, `EstimateEV`.

BlackjackGame.wl — the `DynamicModule` front-end. Holds all reactive state (deck, multi-hand player, dealer hand, bankroll, bet, session history, running count, coach cache), all handlers (`doHit`, `doStand`, `doDouble`, `doSplit`, `startRound`, `advanceHand`, `playoutDealerAndSettle`), and all widgets (cards, badges, chip buttons, openers, session plot).

The only non-obvious idiom is that all handler functions are declared in the `DynamicModule`'s local-variables list. That puts their `DownValues` in the same persistent dynamic context as the state symbols they mutate; otherwise Button actions silently no-op after the first click because the front-end holds stale references to temporary `Module` symbols that have been garbage-collected.

8. Play the game

The notebook loads the two packages from the parent directory of its own folder (that is, the directory containing `Blackjack.wl` and `BlackjackGame.wl`). Evaluate the cell below; a live playable game will appear underneath.

In[ ]:=
```
SetDirectory[NotebookDirectory[]];
packDir = ParentDirectory[NotebookDirectory[]];
Get[FileNameJoin[{packDir, "Blackjack.wl"}]];
Get[FileNameJoin[{packDir, "BlackjackGame.wl"}]];
```

In[ ]:=
```
BlackjackGame[]
```

How to play:

• Adjust the bet using the **$5 / $25 / $100** chip buttons, or **Clear** / **All in**.

• Click **Deal** (or press N) to start a round. The dealer shows one card; the other is face-down until you stand.

• On your hand, choose one of:

Hit (H) — take another card;

Stand (S) — end your turn;

Double (D) — available on the first two cards; doubles the bet and deals one card;

Split (P) — available on any pair of equal-value cards; creates a sister hand with an equal bet.

• When every hand has stood or busted, the dealer plays and the round settles. A natural blackjack pays 3 : 2; any other win pays 1 : 1; push returns the bet; bust or loss forfeits the bet.
• The *Options* opener lets you toggle H17 / S17, set the number of decks (1 / 2 / 4 / 6 / 8), adjust the cut-card penetration and the dealer-draw animation speed.
• Open *Coach* to see the basic-strategy recommendation and request a Monte-Carlo EV estimate for the current state.
• Open *Shoe & card count* for the live Hi-Lo running / true count.
• Open *Session* for a cumulative bankroll plot.

9. Standalone use (without the notebook)

The game runs equally well outside this notebook, in any Wolfram Front End that supports **`DynamicModule`**. Clone the repository (or just download the two **`.wl`** files from the **`Wolfram/`** folder) and evaluate:

```
Get["/path/to/Blackjack.wl"]
Get["/path/to/BlackjackGame.wl"]
BlackjackGame[]
```

Or, from a shell (requires **`wolframscript`** and a Wolfram Front End):

```
wolframscript -file /path/to/BlackjackGame.wl
```

which opens a notebook window with the game embedded. The **`tests.wls`** file in the same folder runs a 66-assertion suite covering deck construction, ace-aware scoring (including a permutation-invariance property), bust/blackjack/soft predicates, S17 and H17 dealer policy, **`PlayDealer`** termination (random trials), round resolution, double/split eligibility, payout math, Hi-Lo count, basic-strategy spot checks, and Monte-Carlo EV ordering on two known positions.

10. References

• E. Thorp, *Beat the Dealer*, Vintage, 1966. The original card-counting book; introduced the first ten-count system.
• R. Baldwin, W. Cantey, H. Maisel & J. McDermott, "The Optimum Strategy in Blackjack", *Journal of the American Statistical Association* **51**(275), 429-439 (1956). The basic-strategy table, derived by hand at Aberdeen Proving Ground.
• P. Griffin, *The Theory of Blackjack*, Huntington Press, 6th ed., 1999. The canonical modern reference on rule variations and composition-dependent strategy.
• J. L. Kelly, "A New Interpretation of Information Rate", *Bell System Technical Journal* **35**(4), 917-926 (1956). Optimal betting under a known edge.
• Repository (source, tests, math write-up, HTML twin): github.com/mthiel74/blackjack.